

LABSHEET: 1 Text Summarization:

Before running the code do install the necessary models:

```
!pip install google.generativeai
```

EXP1.1: TEXT MODELS:

```
import google.generativeai as genai import os

GEMINI_API_KEY = 'Paste your Api_key'      # API key should be a string

genai.configure(api_key=GEMINI_API_KEY)

os.environ['GOOGLE_API_KEY'] =

GEMINI_API_KEY # Initialize the model

model = genai.GenerativeModel('gemini-3.5-flash') #

Send a simple prompt

response = model.generate_content("You will be provided with statements, and your task is to convert them to
standard English. Here is the statement: She no went to the market.")

print(response.text)
```

EXP 1.2:

```
import google.generativeai as genai import os

GEMINI_API_KEY = 'Paste your Api_key'      # API key should be a string

genai.configure(api_key=GEMINI_API_KEY)

os.environ['GOOGLE_API_KEY'] =

GEMINI_API_KEY # Initialize the model

model = genai.GenerativeModel('gemini-3.5-flash') #

Send a simple prompt

response = model.generate_content('Hello! What can you do? List 5 capabilities.') print(response.text)
```

EXP 1.3:

```
import google.generativeai as genai
```

```
model = genai.GenerativeModel('gemini-3.5-  
flash') meeting_minutes = ""
```

Car Manufacturing Company — Board of Directors Meeting

1. CEO Bharath Thippireddy presented quarterly sales figures showing 12% growth.
2. Cost reduction initiatives: shift to renewable energy in 3 plants by Q3.
3. CFO reported revenue of INR 450 Cr, profit margins at 18%.
4. Cash flow analysis: positive outlook; capex approved for new EV line.
5. Board approved INR 50 Cr shareholder dividend for FY

2025. "

```
prompt = f'Summarize the following board meeting minutes in exactly 4 bullet points. Be  
concise.\n\n{meeting_minutes}'
```

```
response = model.generate_content(prompt)
```

```
print('=== MEETING SUMMARY ===')
```

```
print(response.text)
```

LABSHEET:2

EXP 2.1: Sentimental Analysis

```
import google.generativeai as genai
```

```
model = genai.GenerativeModel('gemini-3.5-flash')
```

```
reviews = [
```

```
    'The product is absolutely amazing! Best purchase I ever made.',
```

```
    'Terrible quality. Broke after one day. Very disappointed.',
```

```
    'It is okay. Nothing special but gets the job done.',
```

```
    'Shipping was delayed but the product itself is decent.',]
```

```
for i, review in enumerate(reviews, 1):
```

```
    prompt = f'Classify the sentiment of this customer review as Positive, Negative, or Neutral. Reply with  
one word only.\n\nReview: {review}'
```

```
    response = model.generate_content(prompt)
```

```
    sentiment = response.text.strip()
```

```
    print(f'Review {i}: {review[:60]}...')
```

```
    print(f'Sentiment: {sentiment}\n')
```

EXP 2.2: Code Explanation:

```
import google.generativeai as genai
```

```

import os

# Configure the API key within this cell for explicit access

GEMINI_API_KEY = 'Paste your Api_Key'    # API key should be a string

genai.configure(api_key=GEMINI_API_KEY)

os.environ['GOOGLE_API_KEY'] = GEMINI_API_KEY

model = genai.GenerativeModel('gemini-3.5-flash')

code_snippet = """

def fibonacci(n):

    if n <= 1:

        return n

    return fibonacci(n-1) + fibonacci(n-2)

"""

prompt = f'Explain what this Python code does in simple terms for a beginner. Include: what it does, how it works step by step, and one example.\n\nCode:\n{code_snippet}'

response = model.generate_content(prompt)

print('=== CODE EXPLANATION ===')

print(response.text)

```

Lab sheet 3: Image and Text Generative AI Exercises

Ex 3.1: Text-to-Image using Stable Diffusion XL (Local GPU Generation)**

Ex 3.2: Image-to-Text using Gemini Vision (URL input)**

Install necessary libraries for the exercises

```
!pip install -U diffusers transformers accelerate google-generativeai pillow requests python-dotenv
```

EXP 3.1 -Text-to-Image using Stable Diffusion XL (Local GPU Generation)

```

import os

import requests

from PIL import Image

import io

from IPython.display import display

import torch

from diffusers import DiffusionPipeline

```

```

from dotenv import load_dotenv

# Load environment variables from .env file

load_dotenv()

# Handle environment compatibility (Colab vs local Jupyter)

try:

    from google.colab import userdata

    HF_TOKEN = userdata.get('HF_TOKEN')

except ImportError:

    HF_TOKEN = os.environ.get('HF_TOKEN')

# If token is not set, prompt the user for it

if not HF_TOKEN:

    import getpass

    HF_TOKEN = getpass.getpass("Enter your Hugging Face token here: ")

pipe_sdxl = None

# Load the model directly from HuggingFace Hub

try:

    pipe_sdxl = DiffusionPipeline.from_pretrained(

        "stabilityai/stable-diffusion-xl-base-1.0",

        torch_dtype=torch.float16, # Use float16 for better performance and memory

        use_safetensors=True,

        token=HF_TOKEN,

        variant="fp16", )

    # Move model to GPU if cuda is available

    if torch.cuda.is_available():

        pipe_sdxl.to("cuda")

        print("Stable Diffusion XL model loaded successfully on GPU for local generation.")

    else:

        pipe_sdxl.to("cpu")

        print("Stable Diffusion XL model loaded successfully on CPU for local generation.")

except Exception as e:

    print(f"Failed to load Stable Diffusion XL model locally: {e}")

def generate_image_sdxl(prompt, filename=None):

```

```

print(f'Generating with SDXL: {prompt}')

if pipe_sdsl:

    images = pipe_sdsl(prompt, num_inference_steps=25).images

    image = images[0]

    display(image) # Show inline

    if filename:

        image.save(filename)

        print(f'Saved as {filename}')

    return image

else:

    print("SDXL model not loaded. Skipping local generation.")

    return None

# Test SDXL prompts

generate_image_sdsl('watercolor painting of Zurich city with reflections in water, golden hour',
'zurich.png')

generate_image_sdsl('futuristic smart city in India with solar panels and flying vehicles, digital
art')

```

Ex 3.2 — Image-to-Text using Gemini Vision (URL input)

```

import google.generativeai as genai

import requests

from PIL import Image

from io import BytesIO

from IPython.display import display

import os

from dotenv import load_dotenv

# Load environment variables from .env file

load_dotenv()

# Configure the API key

```

```
try:

    from google.colab import userdata

    GEMINI_API_KEY = userdata.get('GEMINI_API_KEY')
except ImportError:

    GEMINI_API_KEY = os.environ.get('GEMINI_API_KEY')

# Fallback to hardcoded key if not found in environment
if not GEMINI_API_KEY:

    GEMINI_API_KEY = 'Enter your API key'

genai.configure(api_key=GEMINI_API_KEY)

os.environ['GOOGLE_API_KEY'] = GEMINI_API_KEY

model = genai.GenerativeModel('gemini-2.5-flash')

# Load image from URL

image_url = 'https://images.unsplash.com/photo-1597431783670-205a592f954e?w=800'

img_bytes = requests.get(image_url).content

image = Image.open(BytesIO(img_bytes))

display(image)

# Ask Gemini to describe it

response = model.generate_content(['What is in this image? Describe in detail.', image])

print("\nImage Description:")

print(response.text)
```

LABSHEET 4: Blood Report Analysis using Gemini Vision

##Users can take any sample Blood report image of choice and use it

Before running the code do install the necessary models:

!pip install -U google-genai pillow

```
from google import genai from PIL import Image import os

API_KEY = "Enter Your API KEY here" IMAGE_PATH = "labtest .jpeg"

client = genai.Client(api_key=API_KEY) if not os.path.exists(IMAGE_PATH):

raise FileNotFoundError(f"Image not found: {IMAGE_PATH}") image =
Image.open(IMAGE_PATH)

print("Fetching available Gemini models...\n") available_models = [m.name for m in
client.models.list()] preferred_models = [

"models/gemini-3.5-flash", "models/gemini-3.1-flash-image", "models/gemini-3.1-flash-lite",
"models/gemini-3.1-pro-preview", "models/gemini-flash-latest", "models/gemini-pro-latest",
"models/gemini-2.0-flash", "models/gemini-2.0-flash-001", "models/gemini-2.0-flash-lite",
"models/gemini-2.0-flash-lite-001"

]

model_name = None

for model in preferred_models: if model in available_models:

model_name = model break

if model_name is None:

print("\nNo compatible model found.") print("Available Models:")

for m in available_models: print(m)

exit()
```

```
print(f"Using Model: {model_name}\n") prompt = ""
```

You are an expert medical laboratory report analyzer. Analyze the uploaded blood report carefully.

Return your response in the following format.

===== PATIENT REPORT SUMMARY

=====

1. Extract every laboratory test into a markdown table. Columns:

- Test Name
- Patient Value
- Normal Range
- Status (Low / Normal / High)

2. Explain every abnormal value.

3. Mention possible causes.

4. Mention possible symptoms.

5. Suggest foods that may help.

6. Suggest lifestyle improvements.

7. Mention whether immediate doctor consultation is recommended.

8. Give an overall health summary.

9. Mention that this analysis is AI-assisted and not a medical diagnosis. ""

```
print("Analyzing report...\n") try:
```

```
response = client.models.generate_content( model=model_name,
```

```
contents=[prompt, image]) print("=" * 80) print(response.text)
```

```
print("=" * 80)
```

```
with open("blood_report_analysis.txt", "w", encoding="utf-8") as f: f.write(response.text)
```

```
print("\nAnalysis saved to: blood_report_analysis.txt") except Exception as e:  
print("\nError while generating response:\n") print(e)
```